

Fabric Lifecycle Agents — repository structure

Three layers, three places. Microsoft's skills say **how**; the firm's standards say **what good looks like**; the client's overrides say **what's different this time**. Precedence runs client > firm > Microsoft.

☐ Microsoft — vendored, read-only ☐ Firm — standards, orchestrators, enforcement ☐ Client — one folder per engagement (really the client's repo) ☐ Plumbing — packaging, CI, evals

- fabric-lifecycle-agents/
 - vendor/
 - skills-for-fabric/
 - plugins/powerbi-authoring/skills/
 - semantic-model-authoring/
 - powerbi-report-*/ (4 skills)
 - skills/
 - semantic-model/SKILL.md
 - report/SKILL.md
 - standards/
 - SKILL.md
 - semantic-model.md
 - report.md
 - naming.yaml · bpa-rules.json
 - engagement-workflow/
 - SKILL.md · templates/
 - agents/
 - modeler.md · reviewer.md · report-reviewer.md · deployer.md
 - hooks/
 - hooks.json · pre-tool-use.sh · post-edit-tmdl.sh
 - commands/
 - new-engagement.md · review.md · sync-upstream.md
 - scripts/
 - lint-tmdl.py · check-report-bindings.py · sync-upstream.sh
 - clients/
 - _template/
 - acme-demo/
 - engagement.yaml · overrides.md · glossary.md
 - requirements.md · spec.md · reviews/ · deployments/
 - models/ Sales.SemanticModel/ · Sales Performance.Report/
 - evals/
 - .github/workflows/
 - .claude-plugin/
 - README.md · CHANGELOG.md · .gitmodules · .gitignore

the firm's repo — installed as a plugin on the consultant's machine

git submodule, pinned to a commit, never edited

Microsoft's how-to for models — used by the semantic-model orchestrator
planning · design · authoring · management — Microsoft's how-to for reports
everything the firm says
ORCHESTRATOR, model domain — routing only, the skill consultants invoke
ORCHESTRATOR, report domain — routing only

index + the one table of overrides of Microsoft guidance
MOD-* modelling rules, DAX-* rules, RLS patterns
RPT-* rules, review checklist, firm theme
the machine-checkable subset — used by hooks and CI
phases, gates and templates shared by every domain
requirements, model-spec, review-report, handover
subagents the orchestrators delegate to
reviewers are read-only; deployer needs a passing review + prod token
enforcement, runs whether or not the agent read the rules
blocks vendor/ writes, MCP writes, .pbix, un-tokened prod deploys; lints after every edit
slash commands

plain Python/bash the hooks, evals and CI call

on a real engagement this lives in the CLIENT's repo, not here
what /new-engagement copies
committed demo engagement, used by the evals
config, CO-* client overrides, business terms
the phase artefacts — the engagement's evidence trail
PBIP items: TMDL model and PBIR report — diffable text, the deliverable
fixtures that must fail, scenarios that must pass; run.sh runs in CI
upstream-sync (weekly PR) · evals (every PR) · release (on tag)
plugin.json + marketplace.json — how consultants install it

What each folder is for

The rule for deciding where something goes: if Microsoft wrote it, `vendor/`. If the firm decided it, `skills/standards/`. If one client asked for it, `clients/<code>/overrides.md`. If it must be guaranteed rather than followed, `hooks/`.

Folder	Owner	What it is	When you touch it
vendor/skills-for-fabric/	Microsoft	Microsoft's skills-for-fabric repo as a git submodule pinned to one commit. The agent reads the semantic-model and report skills from here for mechanics: TMDL syntax, MCP calls, REST deployment, PBIR file layout.	Never. A weekly GitHub Action bumps the pin and opens a PR; a human merges. Hooks block any write under this folder.
skills/semantic-model/, skills/report/	Firm	One orchestrator per domain. Each is a short SKILL.md whose description wins the routing match for its domain and whose body says: read the client config, load the layers in precedence order, which subagent does what, the output format. No rules live here.	When adding a domain (copy one), or when the routing or gates change.
skills/standards/	Firm	Everything the firm says. SKILL.md is the index and carries the single table of places where the firm overrides Microsoft. semantic-model.md and report.md are the rulebooks (MOD-*, DAX-*, RPT-*). naming.yaml and bpa-rules.json are the rules a script can check.	This is where practice standards are written and changed. A rule that contradicts Microsoft gets a row in the overrides table.
skills/engagement-workflow/	Firm	The phases (discovery → spec → build → review → deploy → handover, and the report track), the artefact each must produce, the gate to leave it, and the templates.	When the engagement process changes.
agents/	Firm	Subagent definitions: modeler builds, reviewer and report-reviewer are read-only and write the review that pins model_commit, deployer promotes between environments behind the gates.	Rarely; the review and deploy procedures live here.
hooks/	Firm	Scripts Claude Code runs before every tool call and after every edit. They enforce rather than advise: block writes to vendor/, Modeling-MCP writes, .pbix, prod deploys without today's token; lint TMDL and run the report binding check after each edit.	When a rule needs to become a guarantee rather than an instruction.
commands/, scripts/	Firm	/new-engagement, /review, /sync-upstream; and the plain scripts the hooks, evals and CI call (lint-tmdl.py, check-report-bindings.py, sync-upstream.sh).	Occasionally.
clients/	Client	One folder per engagement: engagement.yaml (workspaces, environments, method), overrides.md (CO-* exceptions to firm rules), glossary.md, the phase artefacts, and the PBIP items — the TMDL model and PBIR report. _template/ is copied by /new-engagement; acme-demo/ is the committed sample.	Constantly, on an engagement — but in the client's own repo. Only the template and the demo are committed here; everything else is git-ignored.
evals/, .github/workflows/, .claude-plugin/	Plumbing	Fixtures that must fail and scenarios that must pass; the three workflows (weekly upstream sync, evals on every PR, release on tag); the plugin and marketplace manifests consultants install from.	When Microsoft changes something and a scenario needs adding, or on release.

How one request flows. Consultant types /semantic-model add a YTD measure for acme → the **orchestrator** loads clients/acme/overrides.md, skills/standards/semantic-model.md and Microsoft's semantic-model-authoring, in that precedence → the **modeler** edits clients/acme/models/Sales.SemanticModel/.../_Measures.tmdl (never a live model: MOD-00) → the **post-edit hook** lints it and reports DAX-04 findings back → the **reviewer** later writes a review that records the commit → the **report** domain builds against that commit.